

A Formal Analysis of the Universal Digital Identity Framework (UDIF)

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation, Canada

Abstract. The Universal Digital Identity Framework (UDIF) is a post-quantum identity and object-custody architecture built on hierarchical certificates, capability-bounded authorization, canonically encoded state containers, Merkle-committed registries, anchored audit logs, and a post-quantum authenticated transport. This paper presents a formal security analysis of UDIF under standard game-based definitions and under the assumptions made for its constituent primitives, including ML-DSA or SLH-DSA signatures, ML-KEM or Classic McEliece key encapsulation, SHA3-family hashes, KMAC-based authentication, and the RCS authenticated-encryption channel supplied by the underlying QSTP transport. We formalize the UDIF system model, including identity chains, capability tokens, registry and object commitments, anchor-chain propagation, the policy-and-sequencing layer that UDIF operates over QSTP, epoch rekeying for selected branch trunks, and predicate-based query semantics for minimal disclosure across domains. The model also captures the implementation state stores used for certificates, capabilities, treaties, registries, tunnels, and MCEL ledger checkpoints. For each subsystem we state precise security definitions covering certificate-chain authenticity, capability soundness, registry and object integrity, anchor-chain consistency, session confidentiality and integrity inherited from QSTP, forward secrecy of branch trunks against later state compromise, minimal-disclosure query privacy, and treaty-scoped cross-domain containment. The results are deliberately conditional. Forging certificates or escalating capabilities reduces to signature or MAC forgery; equivocating object or registry state, or forking anchored histories, reduces to hash collision, second-preimage, or signature-forgery events; attacks on the encrypted transport reduce to the security of the underlying QSTP handshake and record channel; and attacks that exceed authorized query visibility reduce to commitment binding or capability unforgeability. The analysis is stated in the conventional post-quantum, classical-query model for the protocol adversary unless a stronger quantum-oracle model is explicitly assumed for the sponge functions. Within those boundaries, UDIF provides authenticity, integrity, confidentiality, auditability, forward secrecy for erased trunk epochs, and minimal disclosure relative to the primitive and deployment assumptions.

Keywords: post-quantum identity · capability authorization · Merkle commitments · audit logs · minimal disclosure

1 Introduction

The Universal Digital Identity Framework (UDIF) is a post-quantum identity and provenance architecture designed to provide a unified, cryptographically sound substrate for representing entities, administering authority, managing object custody, and enforcing auditability across diverse domains. UDIF addresses a long-standing gap in digital infrastructure: the absence of a universal, tamper-evident, post-quantum secure foundation

E-mail: contact@qrcscorp.ca (John G. Underhill)

for identity binding, ownership tracking, cross-domain verification, and minimal-disclosure queries. Existing identity and object registries, whether governmental, financial, enterprise, or supply-chain, typically rely on siloed databases, classical cryptography, and non-portable semantics. UDIF replaces these ad hoc structures with a rigorously specified system built on hierarchical certificates, deterministic canonical encodings, Merkle-committed state containers, anchored audit logs, and a PQ-secure transport layer.

At a structural level, UDIF organizes participants into a rooted hierarchy of authorities. A Root Authority defines the cryptographic suite of the domain and issues certificates to Branch Controllers. A Branch Controller may issue certificates to subordinate Branch Controllers or to Group Controllers, and a Group Controller issues certificates to its leaf User Agents (the client role in the implementation). User Agents are the leaf nodes of the hierarchy and the sole owners of objects. Every entity is represented by a canonical encoded certificate signed by its parent, forming a verifiable chain to the Root. Capabilities and access masks constrain each node's authority by binding explicit permissions into certificate and token fields. Objects, registries, membership logs, and transaction logs are maintained as Merkle-committed containers, with periodic Anchor Records relayed upstream to provide continuous, tamper-evident auditability. Communication between UDIF nodes runs over an underlying post-quantum authenticated channel (QSTP), which supplies the mutually authenticated handshake, the cSHAKE-based key schedule, and the RCS-based (or AES-GCM) AEAD record protection. Over this channel UDIF layers a record header, strict sequence, epoch, and time-window controls, and, for long-lived branch trunks, scheduled symmetric ratchet rekeying of the underlying channel. Query operations follow a predicate-based minimal-disclosure model, where entities may learn only Boolean answers or capability-restricted proofs, and cross-domain requests are governed by bilateral treaties.

The goal of this paper is to give a formal cryptanalysis of UDIF under standard game-based definitions and the security properties induced by its construction. We present a functional and adversarial model of UDIF, formalize its certificate and capability system, canonical data model, registry and object commitments, anchored audit chains, transport handshake and ratchet, and predicate-based query semantics. For each subsystem we define the precise security goals, including certificate-chain authenticity, capability soundness and non-escalation, registry and object integrity, anchor-chain consistency, transport confidentiality and integrity, forward secrecy of erased trunk epochs, minimal-disclosure privacy, and cross-domain containment. We prove that these properties hold up to negligible advantage under the cryptographic assumptions underpinning the UDIF primitives, including ML-DSA or SLH-DSA signatures, ML-KEM or McEliece encapsulation, the configured QSTP record channel, SHA3-family hashing, KMAC authentication, and the modeled pseudorandomness or indistinguishability assumptions for sponge-based functions.

The remainder of this paper proceeds as follows. Section 2 fixes notation and cryptographic preliminaries. Section 3 gives a formalization of the UDIF system model, including entities, certificates, capabilities, registries, logs, anchors, transport, and query semantics. Section 4 defines the security properties targeted by UDIF. Section 5 states the cryptographic assumptions used in the analysis. Sections 6 through 9 provide security proofs for certificates and capabilities, registry and anchor integrity, the transport layer, and the query and treaty model. Section 10 gives the composition theorem establishing UDIF's end-to-end guarantees. Section 11 discusses adversarial work factors and recommended parameter sets. Section 12 aligns the formal model with the implementation state machine. Section 13 analyzes limitations and deployment considerations, and Section 14 concludes.

1.1 Background and Motivation

Modern digital infrastructure lacks a universal, cryptographically enforced identity substrate capable of representing individuals, institutions, object registries, and asset provenance with strong authenticity, minimal disclosure, and tamper-evident auditability. Identity systems such as SSNs, KYC registries, directory services, and certificate-based authentication are fragmented, centralized, or limited in scope, and they lack cryptographic binding between entities and the objects they own or manage. Provenance systems in finance, supply chains, and digital assets rely on mutable databases or ledger-like constructs that expose too much information or lack formal guarantees. As digital interactions increasingly span jurisdictions and independent administrative domains, a need arises for a verifiable identity and custody system that preserves privacy, enforces least privilege, and remains secure against classical and quantum adversaries.

UDIF addresses this need by defining a polymorphic identity and object container model equipped with post-quantum certificates, capability-scoped authorization, canonical encodings, Merkle-committed registries, anchored audit propagation, and a PQ-secure transport. The system is designed for deployment in adversarial and cross-domain environments where passive observation, active interference, compromised nodes, treaty misuse, and cryptanalytic threats must all be assumed.

1.2 Overview of the UDIF Construction

UDIF organizes participants into a hierarchical trust model rooted at a single Root Authority. Every entity holds a certificate containing a suite identifier, role, serial number, issuer serial, validity window, public key, capability and access mask fields, and a parent's signature. Capabilities are implemented as KMAC-tagged bitmaps that constrain allowable verbs and scopes. All structures in UDIF, including certificates, objects, registries, logs, anchors, queries, and treaties, are encoded canonically using encodings with strict ordering and deterministic minimal-length.

Objects are represented as canonical records containing serials, type codes, creator bindings, attribute Merkle roots, ownership assignments, and signatures. Each UA maintains a registry of object digests organized into a Merkle tree, with the registry root periodically committed to membership logs. GCs and BCs maintain membership and transaction logs, which are folded into periodic Anchor Records containing Merkle roots and counters. Anchors propagate upward through the hierarchy, forming a tamper-evident audit chain to the Root.

Sessions between UDIF nodes run over an underlying post-quantum authenticated channel, the Quantum Secure Tunnelling Protocol (QSTP), whose mutually authenticated handshake establishes shared keys from KEM encapsulation, digital signatures, nonces, and a transcript hash, and whose cSHAKE-based key schedule drives an RCS-based (or AES-GCM) AEAD with per-record protection. On top of this channel UDIF binds a record header carrying a flag field, a strictly monotonic sequence number, a timestamp, an epoch counter, and a suite identifier, and enforces sequence, epoch, and timestamp-window checks on receipt. Long-lived branch trunks are periodically rekeyed by scheduling the underlying channel's symmetric ratchet, which provides forward secrecy across epochs by deriving fresh keys from the prior channel state and erasing superseded key material.

Queries operate under a minimal-disclosure predicate model. Predicate families include existence checks, ownership binding, attribute-bucket membership, and registry membership proofs. Queries require explicit capabilities and are logged and anchored. Cross-domain queries flow through bilateral treaties that constrain which predicate families may be forwarded and audited in both domains.

1.3 Security Objectives of UDIF

UDIF targets the following informal security objectives:

- **Authenticity of certificates and identities:** Only valid parent-signed certificates may appear in the hierarchy, and no adversary can forge or elevate authority.
- **Capability soundness and non-escalation:** An entity's permissions cannot exceed those granted by its parent or authorized capability tokens.
- **Object and registry integrity:** Objects, registry states, and ownership histories cannot be equivocated or rolled back without detection.
- **Anchor-chain consistency and auditability:** Logs and anchor histories cannot be forked or manipulated without breaking cryptographic assumptions.
- **Transport confidentiality and integrity:** All messages exchanged between UDIF nodes remain confidential and tamper-evident under active attacks.
- **Forward secrecy of long-lived trunks:** Long-lived branch trunks are periodically rekeyed through the underlying channel's symmetric ratchet, so that compromise of current key material does not expose traffic from prior epochs.
- **Minimal disclosure:** Adversaries learn only predicate outcomes authorized by capabilities and cannot infer hidden state or attributes.
- **Cross-domain containment:** Treaty-mediated interactions reveal nothing beyond the explicitly permitted predicate families.

These objectives motivate the formal definitions and proofs presented in the remainder of the paper.

1.4 Roadmap

The remainder of this paper is structured to isolate the cryptographic components of UDIF, define their security properties, and prove them under standard assumptions. Section 2 introduces notation, formal models, and the interfaces for the primitives that UDIF relies upon. Section 3 presents a complete formal system model of UDIF, including certificates, capabilities, registries, logs, anchor propagation, the transport channel, and query semantics. Section 4 defines the security goals for each subsystem. Section 5 outlines the hardness assumptions and models used in the proofs. Sections 6, 7, 8, and 9 contain the main technical results, giving security proofs for the certificate and capability system, the registry and anchor architecture, the transport layer, and the query and treaty model. Section 10 demonstrates how these component results compose into end-to-end guarantees. Section 11 discusses parameter choices and adversarial work factors. Section 12 aligns the formal model with the implementation state machine. Section 13 presents limitations and deployment considerations, and Section 14 concludes.

2 Preliminaries and Notation

This section introduces notation, probability models, and cryptographic interfaces used throughout the analysis. All security statements are with respect to a security parameter $\lambda \in \mathbb{N}$ and adversaries with probabilistic polynomial time complexity unless otherwise stated.

2.1 Basic Notation

Let $\{0, 1\}^n$ denote the set of all bit strings of length n . Let $\{0, 1\}^*$ denote the set of all finite bit strings. For a string x , the notation $|x|$ denotes its bit length. For bit strings x and y , the symbol $x||y$ denotes concatenation. For a probabilistic algorithm A , the notation $y \leftarrow A(x)$ denotes running A on input x and obtaining an output sampled from its internal randomness.

For a set S , the notation $x \xleftarrow{\$} S$ denotes sampling x uniformly at random from S . A function $\epsilon(\lambda)$ is negligible if for every polynomial $p(\lambda)$ there exists λ_0 such that $\epsilon(\lambda) < 1/p(\lambda)$ for all $\lambda > \lambda_0$.

For an adversary $\mathcal{A}$ and experiment Exp , the advantage of $\mathcal{A}$ is written

$$\text{Adv}_{\text{Exp}}^{\mathcal{A}}(\lambda)$$

which denotes the probability that $\mathcal{A}$ wins the experiment, minus the probability that a trivial random guess would succeed where applicable.

We use $\text{poly}(\lambda)$ to denote any function bounded by a polynomial in λ . All algorithms and adversaries are assumed to run in time $\text{poly}(\lambda)$.

2.2 Cryptographic Primitives

This subsection defines the abstract interfaces and security notions for the primitives used in UDIF. The proofs refer only to these interfaces and not to specific implementations.

Key Encapsulation Mechanism (KEM). A KEM consists of algorithms

$$(\text{KeyGen}, \text{Encaps}, \text{Decaps})$$

where **KeyGen** outputs a public key and a secret key, **Encaps** outputs a ciphertext and a shared secret under the public key, and **Decaps** recovers the shared secret from the ciphertext under the secret key. The KEM is IND-CCA secure if no probabilistic polynomial time adversary with decapsulation oracle access can distinguish the real shared secret from a random value.

Digital Signature Scheme. A signature scheme consists of

$$(\text{KeyGen}, \text{Sign}, \text{Verify})$$

where **Verify** outputs accept or reject. The scheme is EUF-CMA secure if no adversary with adaptive signing oracle access can forge a valid signature on a new message.

Authenticated Encryption with Associated Data (AEAD). An AEAD interface provides

$$\text{Enc}(k, n, a, m) \quad \text{and} \quad \text{Dec}(k, n, a, c)$$

where k is a key, n is a nonce, a is associated data, m is plaintext, and c is ciphertext. The AEAD must be IND-CPA secure for confidentiality and INT-CTXT secure for integrity. Associated data must be authenticated and bound into the ciphertext.

Hash and XOF Functions. UDIF uses the SHA3 family, including SHA3, SHAKE, cSHAKE, and KMAC. These are modeled either in the standard model or as ideal permutations where appropriate. We assume collision resistance, preimage resistance, and indistinguishability properties where required. For a hash function H , the output is written

$$H(x) \in \{0, 1\}^n$$

for a fixed output length n determined by the construction. For XOFs, outputs are variable length.

2.3 Canonical Binary Encoding Model

All UDIF structures are serialized according to the fixed-layout canonical binary encoding defined normatively in the UDIF Specification (Revision 1b, Section 4.3). Rather than restating the byte-level layout in this analysis, we model serialization abstractly while binding it explicitly to the specification.

Encoding Function. Let

$$\text{Enc} : \mathcal{R} \rightarrow \{0, 1\}^*$$

be the canonical serialization function mapping any valid UDIF record $R \in \mathcal{R}$ to its unique byte representation.

The function Enc is defined by the following properties:

- **Determinism.** For every valid record R , the output $\text{Enc}(R)$ is uniquely determined.
- **Injectivity.** If $R_1 \neq R_2$, then $\text{Enc}(R_1) \neq \text{Enc}(R_2)$.
- **Fixed Layout.** Each record type has a predetermined field order with fixed-size fields, except for suite-dependent cryptographic components (e.g., signatures and public keys), whose sizes are uniquely determined by the suite identifier.
- **Little-Endian Integers.** All multi-byte integers are encoded in little-endian form.
- **Delimiter-Free.** No length prefixes, padding, or structural markers are inserted between fields.

Invalid encodings, including incorrect total length, malformed field boundaries, or extraneous trailing bytes, are rejected and do not belong to the domain of Enc .

Signature-Exclusion Rule. For any signed record $R = (\sigma, F_2, \dots, F_n)$ where the signature field σ appears first in canonical layout, we define the signature-excluded encoding:

$$\text{Enc}^*(R) = \text{Enc}(F_2, \dots, F_n).$$

All digital signatures in UDIF are computed over the canonical encoding excluding the signature field itself.

Security Assumption (Encoding Injectivity). We assume that Enc is injective over all valid records. Consequently, any two distinct abstract records produce distinct byte strings. This eliminates encoding malleability and structural ambiguity.

2.4 Domain-Separated Hashing

In the implementation, the domain-separated hash H is instantiated as cSHAKE-256 with a 256-bit (32-byte) output, where the domain label is supplied as the cSHAKE customization (function-name) string rather than prepended to the message. We write this uniformly as

$$H_{\text{label}}(x) = \text{cSHAKE-256}(x; \text{label}),$$

and, where the distinction is immaterial to the argument, abbreviate it $H(\text{label} \| x)$. Treating H as a domain-separated random oracle, the two notations are interchangeable for the reductions in this paper. All invocations of H in UDIF are domain-separated using fixed ASCII labels.

For any label `label` and record R , define:

$$\text{Digest}_{\text{label}}(R) = H_{\text{label}}(\text{Enc}(R)).$$

For signed records, digests used as signature inputs are defined as:

$$\text{SigHash}_{\text{label}}(R) = H_{\text{label}}(\text{Enc}^*(R)).$$

Examples include:

$$\begin{aligned} \text{ObjDigest}(O) &= H_{\text{"UDIF:OBJ-DIGEST:V1"}}(\text{Enc}(O)), \\ \text{CertHash}(C) &= H_{\text{"UDIF:CERT-DIGEST:V1"}}(\text{Enc}^*(C)), \\ \text{TxDID} &= H_{\text{"UDIF:TXID:V1"}}(\text{Enc}(\text{transaction content})). \end{aligned}$$

Domain separation ensures that identical byte sequences used in different contexts cannot be substituted across domains or record types.

2.5 Adversarial Model and Oracles

We analyze UDIF in a multi party setting with adaptive adversaries capable of interacting with UDIF nodes at all layers. The adversary controls the network and schedules all message delivery. The adversary may corrupt nodes, request signatures, decapsulations, and AEAD encryptions or decryptions under controlled conditions, and submit arbitrary queries to the UDIF system according to the capabilities it has obtained.

The environment interacts with UDIF through oracles representing the operations available to entities in the system. These include:

- Certificate issuance oracles for parent nodes.
- Capability issuance and revocation oracles.
- Object creation, update, and transfer oracles.
- Registry and log completion oracles that return state commitments.
- Transport session oracles that simulate the authenticated handshake and record exchange.
- Query oracles for existence, ownership binding, attribute bucket membership, and registry membership.
- Treaty forwarding oracles for cross domain predicate evaluation.

The adversary may adaptively choose its queries and may attempt to cause inconsistencies, equivocations, replay attacks, or unauthorized escalations. The security games defined in later sections capture the limits of what the adversary can learn or influence without breaking the underlying cryptographic assumptions.

3 UDIF System Model

This section provides a complete functional model of UDIF. All security definitions and proofs in later sections are with respect to this model. The model captures the entities, certificates, capabilities, registries, logs, anchor propagation, transport channels, and query mechanics described in the UDIF specification.

3.1 Entities and Roles

Let $\mathcal{R}$ denote the set of Root Authorities. A Root is a unique element per domain and defines the cryptographic suite. Let $\mathcal{B}$ denote the set of Branch Controllers (role `ubc`) and $\mathcal{G}$ the set of Group Controllers (role `ugc`). Roles are fixed at certification time and govern the permitted issuance transitions: a Root issues certificates only to Branch Controllers; a Branch Controller issues certificates to subordinate Branch Controllers or to Group Controllers; and a Group Controller issues certificates only to User Agents. Let $\mathcal{U}$ denote the set of User Agents (the `client` role). UAs are leaf entities and are the only holders of objects. Let $\mathcal{O}$ denote the set of object records. The implementation additionally reserves specialized object-registry and auditor roles; these are outside the certificate-issuance hierarchy analyzed here and are not treated as issuing or owning authorities.

We model Roots, BCs, and GCs as long lived servers that maintain state across sessions. UAs are modeled as clients that initiate authenticated transport sessions with their respective GC. Persistent inter-authority links (GC–BC, BC–BC, and BC–Root) are maintained as long term trunks, with BC–BC trunks subject to periodic ratchet rekeying. All entities have persistent key material and certificate chains.

3.2 Certificates and Identity Chains

A certificate is modeled as a tuple whose fields appear in the canonical serialization order

$$C = (\sigma, pk, s_{\text{issuer}}, s, t_{\text{from}}, t_{\text{to}}, \text{cap}, \pi, \text{role}, \text{suite_id})$$

where:

- σ is the issuer signature, which occupies the first field in canonical layout and is excluded from the signed encoding,
- pk is the suite-dependent verification key,
- $s_{\text{issuer}} \in \{0, 1\}^{128}$ is the issuer serial (the parent certificate serial),
- $s \in \{0, 1\}^{128}$ is the 128-bit certificate serial,
- $t_{\text{from}}, t_{\text{to}}$ are the validity window bounds,
- cap is the embedded 64-bit capability bitmap,
- π is the 64-bit policy word governing the validation rules in force for the certificate,

- `role` and `suite_id` are the entity role and the suite identifier.

The signature field σ is defined over the signature-excluded canonical encoding as

$$\sigma = \text{Sign}_{sk_{\text{issuer}}}(\text{SigHash}_{\text{"UDIF: CERT-DIGEST: V1"}}(C)),$$

and verification succeeds if and only if

$$\text{Verify}_{pk_{\text{issuer}}}(\sigma, \text{SigHash}_{\text{"UDIF: CERT-DIGEST: V1"}}(C)) = 1.$$

Enrollment proceeds through a certificate signing request (CSR). A requesting entity generates a fresh key pair and submits a CSR carrying the requested verification key, serial, validity window, capability bitmap, policy word, role, suite identifier, a nonce, and a timestamp, signed under the requested key as a proof of possession. Before issuing a subordinate certificate, the parent verifies the proof-of-possession signature and accepts the request only if (i) the requested role is a permitted transition from the parent role, (ii) the requested capability bitmap is a subset of the parent capability bitmap (no capability bit is set that the parent does not hold), and (iii) the requested validity window is nested within the parent validity window. The issued certificate binds s_{issuer} to the parent serial.

3.3 Capabilities and Access Masks

Capabilities are delegated permission tokens issued by parent authorities. A capability token has the form

$$\text{Cap} = (d, \tau, \sigma_{\text{iss}}, \sigma_{\text{to}}, q, t, v, \pi)$$

where v is the verbs bitmap, q is the scope bitmap, σ_{iss} and σ_{to} are the issuer and subject serials, t is the expiration time, π is the policy word, and d is the domain-separated digest

$$d = H_{\text{"UDIF: CAP-DIGEST: V1"}}(\text{Enc}(\sigma_{\text{iss}}, \sigma_{\text{to}}, q, t, v, \pi)).$$

The field τ is a KMAC-256 tag computed over the digest d under the issuer's key with the same domain label. The tag is therefore verifiable by parties that hold the issuer key (in deployment, the issuer and its domain controllers); capabilities are issuer-authenticated rather than publicly verifiable. We assume that capabilities cannot be forged without finding a hash collision in d or forging a KMAC tag under the issuer key.

Access masks are embedded in certificates and constrain attribute level visibility. An entity may act only if both its certificate mask and its valid capability set permit the requested predicate. The effective permissions of entity E at time t are the intersection of the certificate bitmap and all unexpired, non-revoked capabilities issued to E .

3.4 Objects and Registries

Objects are identity containers with mutable ownership and state fields.

$$\text{Obj} = (\sigma, h_A, \sigma_O, \sigma_C, \sigma_U, t_c, t_u, f, \tau)$$

where σ_O is a 32-byte identifying serial, τ is a type code, σ_C is the creator certificate serial, h_A is the Merkle root of committed attributes, σ_U is the current owner certificate serial, t_c and t_u are creation and update times, f is a flags field, and σ is the signature of the current owner over the signature-excluded encoding. Ownership transfer is captured by a separate transfer record that is signed by *both* the sending and receiving owners and carries a transaction identifier; on transfer the object is additionally re-signed under the new owner's key, so a completed transfer requires the concurrence of both parties.

Each UA maintains a registry which is a binary Merkle tree over object leaves. A registry leaf encodes

$$(h_O, h_U, \sigma_O, f, t)$$

where h_O is the object digest, h_U is the owner certificate digest, σ_O is the object serial used as a lookup key, f is a flags field, and t is a timestamp. Leaves are sorted lexicographically by h_O . Each leaf is hashed to a leaf digest $H^{\text{"UDIF:REGROOT:V1"}}(\text{Enc}(\text{leaf}))$, internal nodes are computed as $H^{\text{"UDIF:REGROOT:V1"}}(\text{left} \parallel \text{right})$ over a leaf set padded to a power of two, and the registry root h_{reg} is the apex of the tree (with the empty registry mapping to a fixed domain-separated constant). The registry root is committed by the owning entity as a signed registry commitment carrying the root, an owner serial, a state epoch, and a timestamp; these commitments are recorded in the membership log and folded into Anchor Records.

3.5 Logs and Anchor Records

Each GC or BC maintains two append only logs. The membership log records UA and branch lifecycle events such as enrollment, suspension, revocation, capability grants, and registry commits. The transaction log records object creations, transfers, attribute updates, and status changes.

Each log entry is an encoded event with fields for an event code, subject serial, time, optional data, and a signature. Log entries are hashed into Merkle trees. Let h_{mem} and h_{tx} denote the Merkle roots for the membership and transaction logs over a given interval.

An Anchor Record has the form

$$\text{Anch} = (\sigma, h_{\text{mem}}, h_{\text{reg}}, h_{\text{tx}}, \sigma_B, q, t, c)$$

where σ_B is the child serial, q is a sequence number, t is time, h_{reg} , h_{tx} , and h_{mem} are the registry, transaction-log, and membership-log Merkle roots, and $c = (c_{\text{mem}}, c_{\text{reg}}, c_{\text{tx}})$ is the triple of event counters for the three ledgers. The field σ is a signature by the child over the signature-excluded encoding under the "UDIF:ANCHOR:V1" label.

Anchor sequences are 0-indexed, with the genesis anchor at sequence 0. A parent maintains the last accepted anchor sequence for each child and accepts a new anchor only if its signature verifies and its sequence equals the exact expected successor; chain verification further requires that consecutive anchors share the child serial, advance the sequence by exactly one, and carry strictly increasing timestamps. The three ledger roots are derived from the MCEL persistence layer (Section 12.4), whose membership, registry, and transaction ledgers are sealed into blocks and checkpoints prior to anchoring. By induction up the tree, all committed state becomes anchored at the Root.

3.6 Transport and Session Layer

UDIF does not implement its own cryptographic transport. Sessions run over the Quantum Secure Tunnelling Protocol (QSTP), an underlying post-quantum authenticated channel that provides the mutually authenticated handshake (KEM encapsulation, signatures, nonces, and a transcript hash), the cSHAKE-based key schedule that derives directional keys and nonces, the AEAD record protection (RCS with KMAC, or AES-GCM, per suite), and a symmetric ratchet for rekeying. The security of the handshake, key schedule, record encryption, and ratchet is a property of QSTP; this paper treats QSTP as a secure channel and analyzes the policy and sequencing guarantees UDIF adds on top of it.

Over the QSTP channel UDIF defines a fixed 26-byte record header carrying a flag field, a strictly monotonic sequence number, a sender timestamp, an epoch counter, and a suite

identifier. Each outbound message is encoded, prefixed with this header to form a record, and the entire record (header included) is submitted to QSTP for encryption; the header is therefore confidentiality- and integrity-protected as part of the authenticated QSTP payload rather than carried as cleartext associated data. On receipt, after QSTP has decrypted and authenticated the payload, UDIF deserializes the header and accepts the record only if the suite identifier and epoch match the session, the sequence number equals the next expected receive sequence, the sender timestamp lies within a 60-second window of local time, and the inner message type is consistent with the header flag. The receive sequence is advanced only on full acceptance, so a rejected record cannot desynchronize session state.

For long-lived BC-BC trunks, UDIF schedules periodic rekeying by requesting the QSTP symmetric ratchet (nominally hourly, with bounded jitter), advancing the UDIF epoch counter. UA-GC sessions are ephemeral; GC-BC and BC-Root trunks are persistent but are not subject to the periodic trunk ratchet. The UDIF tunnel maintains init, established, and closing states over the QSTP connection; failures in header validation or in the underlying QSTP record protection result in record rejection and, for fatal conditions, session closure.

3.7 Query and Treaty Model

UDIF supports four predicate families. For each UA or object identifier x and predicate p , a query is an encoded structure

$$Q = (\sigma_Q, \alpha, x, p, t, d)$$

where σ_Q is a query identifier, α is a type code, x identifies the subject, p expresses the predicate parameters, t is an optional time anchor, and d is the capability digest used to authorize the query.

The predicate families are:

- existence queries, which determine whether an entity exists under a controller,
- ownership binding queries, which test whether a UA is the current owner of an object,
- attribute bucket queries, which test whether an attribute lies within a predefined bucket,
- registry membership proofs, which produce yes or no results with optional Merkle paths.

A GC evaluates a query only if the subject holds a valid capability that authorizes the predicate type. Responses are signed and report one of three verdicts, { **no** = 0, **yes** = 1, **deny** = 2 }, optionally accompanied by a Merkle membership proof; a **deny** verdict indicates that authorization failed rather than disclosing the predicate outcome. All queries and responses are appended to the membership log.

Peering treaties define cross domain predicate visibility. A treaty is a bilateral agreement between two domains and carries a treaty identifier, the two domain (branch) serials, a scope bitmap specifying allowed predicate families, a validity window, a policy word, and signatures from both domains. Queries permitted by the treaty may be forwarded across a dedicated transport session between the branches. Both sides log the forwarded query and its response. No predicates outside the treaty scope may be evaluated across domains.

4 Security Definitions

This section defines the formal security properties required of UDIF. Each definition is expressed as an experiment between a challenger and a probabilistic polynomial time adversary $\mathcal{A}$. The adversary is given oracle access matching the UDIF system model and may adaptively issue queries. All advantages are defined with respect to the security parameter λ .

4.1 Certificate Chain Authenticity

UDIF requires that no adversary can produce a certificate chain that verifies under the UDIF rules without being issued by the Root hierarchy. The adversary interacts with certificate issuance oracles that allow it to request certificates for entities it controls but does not allow impersonation of parents.

Game $\text{Game}_{\text{cert}}^{\mathcal{A}}$. The adversary may adaptively:

- request new child certificates from any parent it legitimately controls,
- query any public certificate verification procedure,
- request revocations or suspensions for certificates it controls.

Finally, $\mathcal{A}$ outputs a certificate chain Chain^* for some entity E^* . The adversary wins if:

$$\text{VerifyChain}(\text{Chain}^*) = 1, \quad \text{and} \quad \text{Chain}^* \text{ contains a certificate not issued by a legitimate parent.}$$

Certificate Authenticity Advantage.

$$\text{Adv}_{\text{cert}}^{\mathcal{A}}(\lambda) = \Pr[\mathcal{A} \text{ wins } \text{Game}_{\text{cert}}^{\mathcal{A}}]$$

UDIF provides certificate chain authenticity if this advantage is negligible.

4.2 Capability Soundness and Delegation Safety

Capabilities must be unforgeable and must not allow escalation beyond permissions granted by parents.

Game $\text{Game}_{\text{cap}}^{\mathcal{A}}$. The adversary may:

- request capability tokens issued by parents it controls,
- request certificate issuance oracles for its own entities,
- query verification oracles for any capability digest or tag.

Finally, $\mathcal{A}$ outputs a capability token Cap^* for some subject E^* . The adversary wins if:

- the token verifies under UDIF rules, and
- the effective permissions derived from Cap^* exceed the permissions implied by any valid chain of parents.

Capability Soundness Advantage.

$$\text{Adv}_{\text{cap}}^{\mathcal{A}}(\lambda) = \Pr[\mathcal{A} \text{ wins Game}_{\text{cap}}^{\mathcal{A}}]$$

UDIF provides capability soundness if this advantage is negligible.

4.3 Object and Registry Integrity

Object integrity ensures that object states cannot be forked or equivocated. Registry integrity ensures that UA registries cannot be manipulated or contradicted without detection.

Game $\text{Game}_{\text{obj-reg}}^{\mathcal{A}}$. The adversary may:

- create objects under UAs it controls,
- request object updates and registry updates,
- request Merkle proofs and registry commitments.

Finally, $\mathcal{A}$ outputs two distinct encodings of either:

- an object record for the same serial that both verify under the UDIF rules, or
- a registry state and a Merkle proof that contradict an anchored registry root.

The adversary wins if the challenger verifies both views.

Object and Registry Advantage.

$$\text{Adv}_{\text{obj-reg}}^{\mathcal{A}}(\lambda) = \Pr[\mathcal{A} \text{ wins Game}_{\text{obj-reg}}^{\mathcal{A}}]$$

UDIF provides object and registry integrity if this advantage is negligible.

4.4 Anchor Chain Soundness and Auditability

Anchor chain soundness guarantees that no adversary can generate two valid but inconsistent anchor histories for the same child or violate the monotonic sequence property.

Game $\text{Game}_{\text{anch}}^{\mathcal{A}}$. The adversary may:

- observe all anchors produced by honest nodes,
- request anchor verification oracles,
- schedule arbitrary network delivery patterns.

Finally, $\mathcal{A}$ outputs either:

- two valid anchor sequences for the same child with inconsistent Merkle roots, or
- a valid anchor with a non monotonic sequence number.

Anchor Soundness Advantage.

$$\text{Adv}_{\text{anch}}^{\mathcal{A}}(\lambda) = \Pr[\mathcal{A} \text{ wins Game}_{\text{anch}}^{\mathcal{A}}]$$

The anchor system is sound if this advantage is negligible.

4.5 Transport Confidentiality and Integrity

The transport channel must provide confidentiality, integrity, replay resistance, and protection of the UDIF record header. Confidentiality and ciphertext integrity are properties of the underlying QSTP channel; the games below state the end-to-end guarantee for a UDIF record, whose header is carried inside the authenticated QSTP payload.

Confidentiality Game $\text{Game}_{\text{conf}}^{\mathcal{A}}$. The adversary may:

- initiate arbitrary sessions with honest nodes,
- observe QSTP ciphertexts and outer transport metadata, while the UDIF inner record header remains encrypted as part of the QSTP payload,
- request encryptions and decryptions subject to standard IND-CPA restrictions,
- interfere with message ordering and timing.

The adversary chooses m_0 and m_1 of equal length. The challenger flips a bit b , encrypts m_b under an active UDIF session, and returns the ciphertext. The adversary outputs a guess b' . The advantage is

$$\text{Adv}_{\text{conf}}^{\mathcal{A}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Integrity Game $\text{Game}_{\text{int}}^{\mathcal{A}}$. The adversary may adaptively submit QSTP ciphertexts and transport packets. Since the UDIF header is an encrypted inner prefix rather than a cleartext AAD field, the adversary cannot choose a UDIF header independently of the ciphertext except by producing a valid QSTP forgery. It wins if any forged inner record is accepted by an honest node.

UDIF transport is secure if both advantages are negligible.

4.6 Forward Secrecy and Ratchet Scope

Long lived BC to BC trunks are periodically rekeyed through the underlying channel's symmetric ratchet, which provides forward secrecy across epochs: superseded key material is erased, so compromise of the state at one epoch does not expose traffic from earlier epochs. We do not claim post-compromise (future) security for the symmetric ratchet, as it introduces no fresh asymmetric secret; recovery from a state compromise is achieved only by establishing a new authenticated session.

Game $\text{Game}_{\text{fs}}^{\mathcal{A}}$. The adversary may:

- compromise session state at some epoch i ,
- observe all past and future ciphertexts,
- force ratchet initiations.

The challenger derives new keys at epoch $i + 1$ via the ratchet. The adversary wins if it decrypts any ciphertext from an epoch prior to i . Forward secrecy holds if this advantage is negligible.

4.7 Query Minimal Disclosure

A query reveals only Boolean predicate outcomes and optional proofs permitted by capabilities. Hidden state should remain indistinguishable.

Game $\text{Game}_{\text{md}}^{\mathcal{A}}$. The adversary chooses two UDIF worlds W_0 and W_1 that differ only in hidden attributes or registry entries. For all predicate types and all capabilities granted to $\mathcal{A}$, the predicate evaluations in both worlds must be identical. The challenger picks b , instantiates world W_b , and answers all authorized queries. The adversary outputs b' . The advantage is

$$\text{Adv}_{\text{md}}^{\mathcal{A}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

Minimal disclosure holds if this advantage is negligible.

4.8 Cross Domain Containment

Cross domain queries must not reveal any information outside the treaty defined predicate scope.

Game $\text{Game}_{\text{cdc}}^{\mathcal{A}}$. The adversary may:

- establish treaties between domains it controls,
- forward queries across domains,
- observe reply logs and anchor records,
- interact with honest GCs through forward and backward tunnels.

The adversary chooses two worlds that differ only in predicate responses not allowed by the treaty scope. The challenger flips a bit b , chooses one world, and answers all cross domain queries permitted by the treaty. The adversary outputs b' . The advantage is

$$\text{Adv}_{\text{cdc}}^{\mathcal{A}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

UDIF satisfies cross domain containment if this advantage is negligible.

5 Cryptographic Assumptions

The security of UDIF relies on the hardness of several well studied cryptographic problems and on the standard security definitions for the primitives used throughout the system. This section states these assumptions and the cryptographic models used in the proofs.

5.1 Primitive Security

The following assumptions are required for the reductions used in later sections.

IND CCA Security of the KEM. Let $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ be either ML KEM or Classic McEliece. The KEM provides IND CCA security if no probabilistic polynomial time adversary with access to a decapsulation oracle can distinguish the shared secret generated by **Encaps** from a uniform string of the same length except with negligible probability. Formally, for any adversary $\mathcal{A}$,

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \text{negl}(\lambda).$$

EUFCMA Security of the Signature Scheme. Let $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be either ML DSA or SLH DSA. The scheme is existentially unforgeable under adaptive chosen message attack if no adversary with oracle access to **Sign** can produce a valid signature on any message that was not previously submitted. Formally,

$$\text{Adv}_{\text{SIG}}^{\text{EUFCMA}}(\mathcal{A}) = \text{negl}(\lambda).$$

Security of the Transport Channel and AEAD. The transport guarantees of UDIF rest on the security of the underlying QSTP channel. Its AEAD record protection is instantiated, per suite, as RCS with KMAC or as AES-GCM, modeled as an authenticated encryption scheme $\text{AEAD} = (\text{Enc}, \text{Dec})$ that must satisfy:

- IND CPA confidentiality, meaning that ciphertexts reveal no information about plaintexts beyond their length,
- INT CTXT integrity, meaning that no forgery can be accepted by a decrypting party.

For any adversary $\mathcal{A}$,

$$\text{Adv}_{\text{AEAD}}^{\text{IND-CPA}}(\mathcal{A}) = \text{negl}(\lambda) \quad \text{and} \quad \text{Adv}_{\text{AEAD}}^{\text{INT-CTXT}}(\mathcal{A}) = \text{negl}(\lambda).$$

We further assume QSTP provides a secure authenticated handshake and a forward-secret symmetric ratchet; these are established in the QSTP analysis and are used here as channel assumptions.

Collision Resistance of SHA3 and Merkle Trees. For the hash function H used in UDIF (SHA3 256 or cSHAKE 256), no adversary can find distinct inputs x and y such that

$$H(x) = H(y)$$

except with negligible probability. Since registries, logs, and anchor structures use Merkle constructions with domain separated hashes, collision resistance of H implies collision resistance of all Merkle commitments.

Indifferentiability of Sponge Constructions. SHAKE, cSHAKE, and KMAC are modeled as indifferentiable from random oracles or ideal permutations when used in XOF or MAC mode. This supports the reductions used for the ratchet KDF, capability KMAC tags, and authenticated headers.

5.2 Modeling Assumptions

Quantum-adversary scope. The post-quantum claims in this paper are stated in the standard Q1 model: adversaries may perform quantum computation against public values and primitive instances, while protocol oracles are queried classically. Statements involving SHA3, cSHAKE, KMAC, Merkle commitments, and transcript binding use the conventional sponge and random-oracle idealizations available in that model. A Q2 model, in which an adversary obtains superposition access to protocol or hash oracles, is not claimed unless a specific primitive proof supplies that stronger oracle treatment.

The analysis uses the following additional modeling assumptions.

Ideal Permutation Model for Keccak. When required, SHA3 based primitives are modeled using the ideal permutation model where the underlying Keccak permutation is treated as a uniformly random bijection. This assumption is standard for indifferentiability proofs of sponge based constructions.

Random Oracle Model for cSHAKE and KMAC. When used as key derivation functions or MAC generators, cSHAKE and KMAC are treated as random oracles or random oracle like objects. This is consistent with their security analyses and the domain separation rules in UDIF.

Deterministic and Injective Canonical Encoding. The canonical little endian encoding function is assumed to be deterministic and injective over the domain of valid UDIF records. This prevents malleability attacks based on alternate encodings and is required for collision resistance of committed structures.

Merkle Tree Soundness. Merkle trees are assumed to be sound. A valid Merkle proof authenticates exactly one leaf and no adversary can construct inconsistent proofs for different leaves without finding a hash collision.

Deterministic Signature and Certificate Verification Model. Verification procedures for signatures, certificates, logs, and anchors are assumed to be deterministic functions of their inputs. No side channels or timing leakage are modeled.

6 Security of Certificates and Capabilities

This section proves that UDIF certificate chains cannot be forged and that capabilities cannot be constructed or escalated beyond what parent authorities authorize. The proofs rely on the EUF CMA security of the signature scheme and the unforgeability of KMAC tags.

6.1 Certificate Chain Authenticity Theorem

Theorem 1. *Let $\mathcal{A}$ be any probabilistic polynomial time adversary in the certificate authenticity game $\text{Game}_{\text{cert}}^{\mathcal{A}}$. Suppose the signature scheme used for UDIF certificates is EUF CMA secure. Then any adversary that forges a valid UDIF certificate chain that was not issued by the Root hierarchy can be used to construct an adversary that breaks the EUF CMA security of the signature scheme. Therefore,*

$$\text{Adv}_{\text{cert}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\lambda) + \text{negl}(\lambda).$$

6.2 Proof of Chain Authenticity

Proof. The proof follows a hybrid argument. Let $\mathcal{A}$ be an adversary that wins the certificate authenticity game. We construct an adversary $\mathcal{B}$ that breaks the signature scheme.

Hybrid H0. This is the real certificate authenticity game. The challenger issues certificates by signing the canonical serialization/encoding of the certificate fields using the real signing key. The adversary outputs a forged chain.

Hybrid H1. The challenger simulates all certificate issuance using the signature oracle of the EUF CMA challenger. Since signatures are always produced by the oracle on honestly formed messages, the view of $\mathcal{A}$ is identical to the real world.

Hybrid H2. When $\mathcal{A}$ outputs a forged certificate chain, at least one certificate in the chain must contain a signature that the challenger did not request from the signing oracle. Otherwise the certificate must originate from a legitimate parent and cannot violate the chain relation. Hence the bottom certificate contains a new signature on a message not requested previously.

Conclusion. The adversary $\mathcal{B}$ forwards this signature and the signed message to the EUF CMA challenger as a forgery. Since $\mathcal{B}$ perfectly simulates the environment, the success probability of $\mathcal{B}$ is identical to that of $\mathcal{A}$ up to negligible error. This concludes the reduction. $\square$

6.3 Capability Unforgeability and Non Escalation

Theorem 2. *Let $\mathcal{A}$ be an adversary in the capability game $\text{Game}_{\text{cap}}^{\mathcal{A}}$. Suppose KMAC is a secure MAC under chosen message attack and the signature scheme is EUF CMA secure. Then any adversary that forges an effective capability not implied by parent permissions yields an adversary that either forges a KMAC tag or forges a signature. Therefore,*

$$\text{Adv}_{\text{cap}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_{\text{KMAC}}^{\text{MAC-UF}}(\lambda) + \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\lambda) + \text{negl}(\lambda).$$

Proof. A forged capability must either contain:

- a valid KMAC tag under the issuer key that was never issued by the parent, or
- a reference to a certificate with an inflated capability bitmap not signed by the parent.

The first case gives a direct MAC forgery. We construct an adversary that uses the capability forger to produce a new digest and tag pair not requested during oracle queries.

The second case implies that the adversary has produced a certificate whose embedded permission bits exceed those of its parent. This requires forging a signature on a certificate that the legitimate parent never signed. Hence this gives a signature forgery.

In each case the reduction is straightforward because the canonical encoding of the capability or certificate is deterministic and injective. This allows extraction of the forged message on which the adversary succeeded. Therefore the advantage in forging or escalating capabilities is bounded by the sum of the advantages in forging KMAC tags or signatures, which are negligible. $\square$

7 Security of Objects, Registries, and Anchors

This section analyzes the integrity guarantees provided by UDIF for object records, UA registries, and the anchored audit chain. Each result follows from the collision resistance of the hash function, the deterministic canonical encoding, the EUF CMA security of signatures, and the append only property of logs. Together these components ensure that no adversary can produce conflicting object states, equivocate registry roots, or create inconsistent anchor sequences without breaking one of the underlying assumptions.

7.1 Object Record Integrity

Theorem 3. *Let $\mathcal{A}$ be an adversary in the object integrity game. Suppose the hash function used for object digests and attribute commitments is collision resistant and the signature scheme is EUF CMA secure. Then any adversary that produces two distinct valid object records for the same serial, or produces a valid object record with an invalid creator or owner binding, can be used to either find a hash collision or forge a signature. Therefore,*

$$\text{Adv}_{\text{obj}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_H^{\text{CR}}(\lambda) + \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\lambda) + \text{negl}(\lambda).$$

Proof. An object record includes a serial, type code, creator digest, attribute Merkle root, current owner serial, timestamps, and a signature by the owner. The digest of the object is

$$h_O = H(\text{label} \parallel \text{Enc}(\text{Obj})).$$

A successful forgery must satisfy one of the following:

1. Two different encodings Obj and Obj' yield valid digests for the same serial. Since the encoding is injective, this gives a collision in H .
2. A forged object record binds a creator or owner that never signed the record. This implies that the adversary has produced a valid signature on a message not signed by the legitimate owner, violating EUF CMA security.
3. A forged attribute Merkle root corresponds to a tree that does not match any attribute set produced by the legitimate creator. This again implies a hash collision in either the leaf hashes or internal nodes.

Since these are the only ways to construct a conflicting or invalid object record, the adversary does not win the game except with negligible probability. $\square$

7.2 Registry Consistency and Non Repudiation

Theorem 4. *Let $\mathcal{A}$ be an adversary in the registry consistency game. Assume that the hash function is collision resistant and the membership logs are append only. Then any adversary that produces two distinct registry states for the same UA and same time, or contradicts an anchored registry root using a forged Merkle proof, can be used to find a hash collision or to violate log consistency. Therefore,*

$$\text{Adv}_{\text{reg}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_H^{\text{CR}}(\lambda) + \text{negl}(\lambda).$$

Proof. A UA registry is a binary Merkle tree whose leaves are the canonical, lexicographically sorted object leaves; the root h_{reg} is the apex obtained by hashing each leaf under the REGROOT label and combining sibling nodes pairwise under the same label.

The adversary may attempt the following attacks:

- Produce two Merkle trees with different leaves but identical root values. Since the canonical ordering of leaves is deterministic, this implies a collision in H .
- Produce a Merkle proof for an object that is not in the registry. A proof includes sibling hashes along a path. A false proof must either collide with an existing leaf or recreate an internal Merkle node without knowing the correct child. Either condition yields a collision in H .
- Contradict an anchored root by producing a registry state that does not match the root already sent to the GC and embedded in the latest Anchor Record. Since anchored states are signed and logs are append only, contradicting an anchor requires either forging a signature or generating inconsistent Merkle roots.

Thus any successful attack against registry integrity implies either a hash collision or a break of append only logs, both of which are ruled out by assumption. $\square$

7.3 Anchor Chain Soundness

Theorem 5. *Let $\mathcal{A}$ be an adversary in the anchor soundness game. Assume the hash function is collision resistant and the signature scheme is EUF CMA secure. Then any adversary who constructs two valid but inconsistent anchor sequences for the same child, or any anchor that violates monotonic sequence properties, can be reduced to a collision attack or a signature forgery. Therefore,*

$$\text{Adv}_{\text{anch}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_H^{\text{CR}}(\lambda) + \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\lambda) + \text{negl}(\lambda).$$

Proof. An Anchor Record binds together the registry root, transaction root, membership root, and counters under a child signature. Let two anchors for the same child be

$$\text{Anch}_1 = (\sigma_B, q_1, t_1, h_{\text{reg}}^1, h_{\text{tx}}^1, h_{\text{mem}}^1, c_1, \sigma_1),$$

$$\text{Anch}_2 = (\sigma_B, q_2, t_2, h_{\text{reg}}^2, h_{\text{tx}}^2, h_{\text{mem}}^2, c_2, \sigma_2).$$

The adversary wins if:

- $q_1 = q_2$ but the Merkle roots differ,
- $q_1 < q_2$ yet the roots contradict the append only log semantics,
- the signature on either anchor verifies but does not correspond to a legitimate child,

- the adversary produces a root that matches the parent’s expectations while reflecting inconsistent underlying logs.

Case analysis:

1. If two anchors with equal sequence numbers have different roots, the attacker has constructed two different Merkle commitments with identical signed metadata. This is impossible without forging signatures or finding hash collisions.
2. If the attacker presents a later anchor whose roots contradict earlier Merkle roots, they must contradict the append only nature of logs. The only way to achieve this is to construct a false Merkle root or forge a signature.
3. If the signature verifies but was not produced by the legitimate child, this is an EUF CMA forgery.

Thus, the adversary cannot produce inconsistent anchors without breaking the collision resistance of the hash function or the unforgeability of signatures. $\square$

8 Transport Layer Security

This section establishes confidentiality, integrity, replay protection, ordering guarantees, and forward secrecy for UDIF transport sessions. UDIF runs over the QSTP channel, which supplies the authenticated handshake, AEAD record protection, and symmetric ratchet; the results below show how UDIF inherits these guarantees and how its record header and sequence, epoch, and time-window checks extend them to replay and ordering resistance. The analysis applies to both UA to GC sessions and BC to BC trunk sessions, with the latter subject to periodic ratchet rekeying.

8.1 Handshake Security

Theorem 6. *Let $\mathcal{A}$ be any probabilistic polynomial time adversary against a UDIF session that distinguishes the established session key from random or impersonates a peer. Since UDIF establishes sessions through the QSTP handshake, and the QSTP handshake derives keys from KEM encapsulation and authenticates peers with digital signatures over a bound transcript, any such adversary can be used to break either the IND CCA security of the KEM or the EUF CMA security of the signature scheme underlying QSTP. Therefore,*

$$\text{Adv}_{\text{hs}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\lambda) + \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\lambda) + \text{negl}(\lambda).$$

Proof. The QSTP handshake exchanges nonces, certificates, signatures, and KEM ciphertexts, and derives the session key from the resulting shared secret(s), the exchanged nonces, and a transcript hash of the form

$$\text{IKM} = ss \parallel \text{nonce}_A \parallel \text{nonce}_B \parallel \text{transcript_hash},$$

expanded with cSHAKE. If an adversary distinguishes the derived key from random, then either the KEM shared secret has been distinguished from random, contradicting IND CCA security, or the adversary has impersonated a peer by forging a signature over the bound transcript, contradicting EUF CMA security. Transcript binding eliminates reflection and unknown-key-share attacks. Because UDIF adopts these keys without modification, its sessions inherit the bound. All other paths to advantage lead to negligible probability. $\square$

8.2 Record Confidentiality and Integrity

Theorem 7. *Let $\mathcal{A}$ be an adversary in the UDIF record security game. Suppose the QSTP AEAD record protection is IND CPA and INT CTXT secure. Since each UDIF record, including its header, is carried as the plaintext of a single QSTP record, the record is confidential and unforgeable under chosen ciphertext attacks, even in the presence of adversarial control of network scheduling, including reordering and chosen ciphertext submission. Therefore,*

$$\text{Adv}_{\text{rec}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_{\text{AEAD}}^{\text{IND-CPA}}(\lambda) + \text{Adv}_{\text{AEAD}}^{\text{INT-CTXT}}(\lambda) + \text{negl}(\lambda).$$

Proof. Each UDIF record is formed by prefixing the 26-byte header h to the encoded message m and encrypting the whole record under QSTP:

$$c = \text{Enc}(k, n, h \parallel m),$$

so the header fields (flags, sequence, timestamp, epoch, suite identifier) are inside the encrypted and authenticated payload. If an adversary distinguishes the ciphertext, it breaks IND CPA confidentiality of the AEAD. If it causes an honest node to accept a forged or modified record, it breaks INT CTXT integrity.

Because the header is covered by the AEAD tag, the adversary cannot modify the sequence, epoch, time, or suite fields without detection, and these fields remain confidential on the wire. Chosen ciphertext attacks cannot succeed because decryption rejects any record that fails AEAD verification, and the receiver additionally rejects any record that fails the header constraint checks.

Therefore the advantage of a successful attack is negligible. $\square$

8.3 Replay, Reordering, and Time Window Enforcement

Theorem 8. *Let $\mathcal{A}$ be an adversary that controls the network and attempts replay or reordering attacks. Suppose timestamps are checked within a bounded window and sequence numbers must increase monotonically. Then any adversary that causes an honest node to accept a replayed or reordered record must either forge a valid AEAD ciphertext or break the header authentication. Therefore,*

$$\text{Adv}_{\text{replay}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_{\text{AEAD}}^{\text{INT-CTXT}}(\lambda) + \text{negl}(\lambda).$$

Proof. A UDIF receiver accepts a record only if:

$$\text{seq} = \text{rxsequence}, \quad |\text{now} - \text{utctime}| \leq 60, \quad \text{epoch} = \text{current_epoch},$$

where rxsequence is the next expected receive sequence and is incremented only after full acceptance. and the AEAD tag verifies.

For a replayed record, the sequence number will not equal rxsequence . For a reordered record, the sequence number likewise will not match the next expected value. For an out of window timestamp, the time check fails. Any tampering with these header fields requires forging a valid QSTP record, since the header is inside the authenticated payload. Thus the only winning path for $\mathcal{A}$ is to break INT CTXT integrity, which is negligible by assumption.

Therefore replay and reordering attacks are prevented except with negligible probability. $\square$

8.4 Forward Secrecy of Trunks

Theorem 9. *Let $\mathcal{A}$ be an adversary in the forward secrecy game. The trunk ratchet is the symmetric ratchet of the underlying channel, which on each rekey derives a fresh epoch state from the prior state by a cSHAKE-based key-derivation step*

$$\text{state}_{i+1} = \text{KDF}(\text{state}_i), \quad k_{i+1} = \text{Expand}(\text{state}_{i+1}),$$

and erases state_i and k_i . Modeling the derivation as a one-way pseudorandom function (equivalently, cSHAKE as a random oracle), compromising the state at epoch $i + 1$ does not enable the adversary to recover state_j or k_j for any $j \leq i$. Therefore the advantage in decrypting any earlier-epoch ciphertext satisfies

$$\text{Adv}_{\text{fs}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_{\text{KDF}}^{\text{ow}}(\lambda) + \text{Adv}_{\text{AEAD}}^{\text{IND-CPA}}(\lambda) + \text{negl}(\lambda).$$

Proof. The ratchet advances the channel key by a one-way derivation and discards the previous state and keys. An adversary that compromises state_{i+1} can compute all later states, but recovering state_i from state_{i+1} requires inverting the derivation, contradicting one-wayness (preimage resistance) of the underlying sponge. Earlier-epoch traffic was encrypted under k_j for $j \leq i$, which depend on the erased earlier states; distinguishing or decrypting that traffic therefore requires either inverting the derivation or breaking AEAD confidentiality. Hence past-epoch confidentiality holds up to the stated bound.

We note explicitly that the symmetric ratchet does *not* provide post-compromise (future) security: because no fresh asymmetric secret is introduced at rekey, an adversary holding the current state can follow all subsequent rekeys. Recovery from a full state compromise requires establishing a new authenticated session rather than relying on the ratchet. $\square$

9 Security of Queries and Peering Treaties

This section analyzes the privacy and containment guarantees of the UDIF query system. Queries operate under a predicate based minimal disclosure model, and cross domain forwarding is constrained by bilateral treaties. The results below show that no adversary can learn information beyond the allowed predicates unless it breaks the collision resistance of hash commitments, the unforgeability of capabilities, or the soundness of anchor propagation.

9.1 Minimal Disclosure for Local Queries

Theorem 10. *Let $\mathcal{A}$ be an adversary in the minimal disclosure game. Suppose the hash function used for attribute commitments and registries is collision resistant, and Merkle proofs are sound. Consider two worlds W_0 and W_1 that differ only in hidden attributes or registry entries, and agree on all predicate evaluations permitted by the adversary's capabilities. Then any adversary that distinguishes which world it interacts with can be used to either find a hash collision or produce inconsistent Merkle proofs. Therefore,*

$$\text{Adv}_{\text{md}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_H^{\text{CR}}(\lambda) + \text{negl}(\lambda).$$

Proof. A query response contains only:

$$\text{verdict} \in \{0, 1, 2\}, \quad \text{proof}, \quad \text{signature}.$$

If the adversary distinguishes W_0 from W_1 , then either:

- the predicate outcome differs, which contradicts the game definition, or
- the proof structure reveals information about hidden state.

A proof is a Merkle authentication path. If the adversary detects a difference in paths without a predicate change, the two worlds must use different leaves or internal nodes that hash to the same root. Since the Merkle root is fixed by anchored registry commitments, this implies either:

1. a collision in H , or
2. an invalid proof that contradicts the Merkle construction.

Thus no distinguishing strategy exists except by breaking the hash function. Hence the advantage is negligible. $\square$

9.2 Cross Domain Query Containment

Theorem 11. *Let $\mathcal{A}$ be a cross domain adversary with treaty mediated access to a foreign domain. Suppose capabilities are unforgeable and the anchor system is sound. Then no adversary can learn information outside the predicate families permitted by the treaty scope without forging either a capability token, a certificate, or an anchor. Therefore,*

$$\text{Adv}_{\text{cdc}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_{\text{cap}}^{\text{forge}}(\lambda) + \text{Adv}_{\text{anch}}^{\text{sound}}(\lambda) + \text{negl}(\lambda).$$

Proof. A treaty specifies a scope bitmap indicating the allowed predicate types. The evaluation procedure is:

1. the origin GC validates the caller capability,
2. the GC checks that the predicate type is allowed by the treaty,
3. the GC forwards the query to the treaty peer,
4. the peer evaluates the predicate locally,
5. both sides log the query and response.

If an adversary extracts additional information, then one of the following must occur:

- the adversary submitted a predicate not covered by its capability bitmap,
- the adversary submitted a predicate not covered by the treaty bitmap,
- the foreign GC evaluated a query outside the treaty scope,
- the adversary suppressed or altered a logged predicate without detection.

The first two cases imply forging a capability digest or KMAC tag. The third case implies forging an anchor or altering logs, because query evaluations are logged and anchored in both domains. The fourth implies either breaking collision resistance of the membership log tree or producing a forged anchor.

Thus any violation of containment reduces to either capability forgery or anchor equivocation. Both are negligible by assumption. $\square$

9.3 Auditability of Query Flows

Theorem 12. *Let $\mathcal{A}$ be an adversary attempting to repudiate treaty interactions or suppress evidence of cross domain queries. Suppose the membership logs and anchor chains are append only and collision resistant. Then no adversary can alter or remove evidence of a query, or present an inconsistent audit record across domains, without breaking hash collision resistance or anchor soundness. Therefore,*

$$\text{Adv}_{\text{audit}}^{\mathcal{A}}(\lambda) \leq \text{Adv}_H^{\text{CR}}(\lambda) + \text{Adv}_{\text{anch}}^{\text{sound}}(\lambda) + \text{negl}(\lambda).$$

Proof. Each query produces a log entry in both the origin and treaty peer domains. These entries include:

- a digest of the query,
- a type code,
- the capability reference,
- a verdict,
- a timestamp,
- a signature of the GC or BC.

Later, both logs are aggregated into the Merkle roots of Anchor Records. For the adversary to suppress or alter evidence of treaty interactions, it must either:

1. remove or alter a logged entry while preserving the Merkle root, or
2. produce two anchor sequences with contradictory log roots.

The first case implies a collision in H , because log entries are canonical and the Merkle root must remain unchanged. The second implies an anchor forgery or a break of append only log semantics. Both are ruled out by collision resistance and anchor soundness.

Therefore all treaty interactions are non repudiable and the adversary cannot produce inconsistent audit trails. $\square$

10 Composition and End to End Security

This section provides a system level view of UDIF security and shows how the local guarantees proven in earlier sections compose into global security properties for full UDIF deployments. The intent is to demonstrate that UDIF behaves as a secure identity, authorization, and provenance framework when all components operate together as specified.

10.1 Composition Framework

We analyze UDIF as an interactive multi party protocol executed among Roots, Branch Controllers, Group Controllers, User Agents, and the adversarial environment. The execution model consists of:

- long term state held by Roots, BCs, and GCs,

- short lived sessions between UAs and their GCs,
- long lived trunk sessions between BCs,
- certificate issuance events,
- capability delegation and revocation events,
- object creation, update, and transfer events,
- registry and log updates with periodic anchoring,
- query evaluations with optional treaty forwarding.

Each component has already been shown to satisfy its respective security definition. To compose these guarantees, we use the following framework.

Independence of Subsystems. Certificates, capabilities, registries, anchors, and transport sessions operate with cryptographically independent keys and nonces. No key material is shared across primitives. This ensures that the failure of one subsystem cannot be used to attack another, except through violations already ruled out by reductions.

Canonical Encoding and Domain Separation. All committed structures are encoded with canonical little endian encoding under strict labels. The domain separation labels ensure that outputs of hash and MAC functions cannot be confused or substituted across contexts. This removes cross protocol interactions that commonly complicate composition.

Monotonicity of State. Logs, registries, and anchor chains are append only. The monotonic nature of their Merkle roots ensures that later states cannot contradict earlier states without breaking collision resistance. This provides temporal structure that simplifies global reasoning.

Authenticated Binding of Components. Certificates bind identity, permissions, and suites. Anchors bind logs. The transport layer binds time, sequence, and epoch metadata into the authenticated QSTP record payload. Queries bind predicate semantics to capability tokens. These authenticated bindings enable modular composition proofs.

Adversarial Scheduling and Concurrency. The adversary controls message scheduling but does not break the cryptographic assumptions. UDIF components are sequentially composable since adversarial scheduling cannot remove authenticated bindings or violate monotonicity.

10.2 Main End to End Security Theorem

Theorem 13. *Let $\mathcal{A}$ be any probabilistic polynomial time adversary interacting with a full UDIF deployment under the system model described above. Suppose the signature scheme is EUF CMA secure, the KEM is IND CCA secure, the AEAD construction provides IND CPA and INT CTXT security, the hash function is collision resistant, and Merkle trees are sound. Then the global advantage of $\mathcal{A}$ in violating any UDIF security goal is bounded by the sum of the advantages in breaking these primitives. Formally,*

$$\begin{aligned} \text{Adv}_{\text{UDIF}}^{\mathcal{A}}(\lambda) &\leq \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\lambda) + \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\lambda) \\ &\quad + \text{Adv}_{\text{AEAD}}^{\text{IND-CPA}}(\lambda) + \text{Adv}_{\text{AEAD}}^{\text{INT-CTXT}}(\lambda) \\ &\quad + \text{Adv}_H^{\text{CR}}(\lambda) + \text{negl}(\lambda). \end{aligned}$$

Proof. Every global UDIF failure corresponds to a violation of one of the local security properties proven earlier. We consider the categories of global failures and apply reduction arguments.

Identity or permission failures. Certificate chain authenticity and capability soundness have already been reduced to EUF CMA and MAC unforgeability. Thus any attempt to impersonate an entity, escalate privileges, or inject unauthorized identity material reduces to signature or MAC forgery.

Object or registry failures. Contradictory object states or inconsistent registry views reduce to hash collisions or signature forgeries. All registry states are Merkle committed and included in anchored logs which are append only.

Anchor chain failures. Any equivocation of logs or anchors also reduces to either a hash collision or a signature forgery. The monotonic sequence property enforces unique progression.

Transport failures. Confidentiality and integrity failures reduce to breaking IND CPA or INT CTXT security of the AEAD or IND CCA security of the KEM. Replay and reordering attacks reduce to INT CTXT because all header metadata is authenticated.

Query and treaty failures. Minimal disclosure failures reduce to hash collisions in committed structures. Treaty scope violations reduce to capability or certificate forgeries, or anchor equivocation. All query flows are logged, and logs are anchored, so any suppression or modification of evidence reduces to hash or signature forgery.

Composition. Since each subsystem operates under independent keys and is protected by authenticated outputs, no combined attack provides more advantage than the sum of the advantages for each primitive. All reductions are tight up to negligible loss due to simulation overhead.

Thus the adversary cannot violate any global UDIF security property without breaking at least one underlying primitive. The right hand side bounds the adversarial advantage by the sum of negligible values, completing the proof. $\square$

11 Adversarial Work Factors and Parameter Choices

This section provides concrete security estimates for the cryptographic primitives used in UDIF and discusses how the recommended parameter sets contribute to the overall security margin. Although the formal proofs rely on asymptotic assumptions, a deployment requires concrete work factor estimates against the best known classical and quantum adversaries.

11.1 Work Factor Estimates

UDIF supports two signature families (ML DSA and SLH DSA), two KEM families (ML KEM and Classic McEliece), and a sponge based AEAD (RCS with KMAC, or AES-GCM) with SHA3/cSHAKE commitments. The default build configures the level-5 lattice suite (ML-KEM-1024, ML-DSA-87, RCS-256/KMAC-256, cSHAKE-256), and the implementation additionally provides level-1 and level-3 suites for lower-overhead deployments. The security levels of these primitives are well understood.

ML KEM. ML KEM at level 5 (ML-KEM-1024) targets NIST post-quantum security category 5, comparable to the brute-force resistance of a 256-bit symmetric key. The best known classical and quantum lattice attacks, including primal and dual attacks, are estimated to remain above this target for the standardized parameters; concrete core-SVP estimates are well in excess of the 128-bit post-quantum floor adopted throughout this analysis.

Classic McEliece. Classic McEliece with binary Goppa codes remains resistant to the best known classical and quantum structural and information-set-decoding attacks. Level 5 parameters target category 5 security, and no subexponential quantum attack is known.

ML DSA. ML DSA at level 5 targets NIST security category 5. Its hardness rests on module-lattice problems (MLWE/MSIS); the best known quantum speedups are modest polynomial improvements rather than a Grover-style square-root reduction, and the scheme remains at or above the requirement for long-term identity and audit preservation.

SLH DSA. SLH DSA is a stateless hash-based signature whose concrete security reduces to the collision and preimage resistance of its underlying hash function. For the SHA3/SHAKE-256 based instances, this provides at least 128-bit security against the dominant collision-finding attacks, with preimage resistance higher still. The overall scheme inherits the lower of these bounds.

RCS and KMAC. The RCS AEAD and the KMAC authenticator are built on the Keccak-1600 permutation in capacity-constrained sponge modes. With a 256-bit capacity, these constructions target 128-bit security against collision-style attacks and provide higher resistance against preimage and key-recovery attacks; KMAC and cSHAKE forgery advantages are bounded by their tag and capacity sizes.

Merkle Trees and SHA3 Commitments. Registry and anchor commitments use 256-bit cSHAKE/SHA3 outputs. Collision resistance is approximately 2^{128} and preimage resistance approximately 2^{256} . Because commitment binding rests on collision resistance, the 128-bit collision bound is the governing floor for registry, log, and anchor soundness.

Taken together, the adversarial work factor required to violate UDIF in any subsystem is governed by the smallest applicable bound. Commitment binding (collision resistance of the 256-bit commitment hash) sets a floor of approximately 2^{128} , which coincides with the 128-bit post-quantum target met by the level-5 asymmetric primitives. UDIF therefore meets the standard 128-bit post-quantum security target across all supported families; deployments requiring a higher classical or quantum margin for a specific subsystem should select parameters and commitment widths accordingly.

11.2 Discussion of Suite Configurations

A UDIF suite specifies a complete set of cryptographic primitives selected at compile time. The choice of suite impacts performance, bandwidth, audit durability, and long term cryptographic confidence.

Lattice based suite. An ML KEM plus ML DSA suite yields the smallest certificate and anchor sizes and the fastest handshake operations. This configuration is suitable for high throughput deployments. Its long term quantum security rests on the hardness of module lattice problems.

Hash based suite. An ML KEM plus SLH DSA suite provides maximal conservatism for signatures, since SLH DSA reduces to hash based hardness alone. Signature sizes and verification costs are larger but provide high assurance for domains requiring multi decade audit durability.

Code based suite. A McEliece plus ML DSA suite offers very high security margins for KEM operations at the cost of larger public keys. This configuration suits deployments where bandwidth is abundant and the long term failure probability must be minimized.

AEAD and hash families. All suites use either AES-256-GCM or RCS with KMAC for AEAD and a 256-bit cSHAKE/SHA3 output for commitments. These primitives balance performance, security, and implementation simplicity and remain conservative under both classical and quantum models.

Tradeoffs between suites do not affect the validity of earlier proofs. All supported suites meet the 128-bit post-quantum security target; the level-5 suites target NIST category 5 for their asymmetric components, while commitment binding is governed by the 128-bit collision resistance of the 256-bit commitment hash common to all suites.

12 Implementation Alignment and State-Machine Completeness

This section aligns the formal model with the implementation mechanisms exposed by the UDIF source tree. The codebase separates the protocol into certificate, capability, treaty, object, registry, anchor, tunnel, dispatch, entity-state, storage, and MCEL manager modules. The formal analysis therefore treats UDIF not merely as an abstract certificate hierarchy but as an executable state machine whose security properties depend on typed stores, role-gated dispatch, strict record validation, and auditable persistence.

12.1 Typed State Stores

The implementation maintains explicit certificate, capability, and treaty stores. A certificate store maps serial numbers to certificates and a status value. Active, suspended, revoked, and expired states are not merely annotations: they are validation inputs. A revoked certificate is terminal, a suspended parent blocks an otherwise active child, and recursive chain validation rejects a child whose issuer has become invalid. A capability store maps capability digests to capability records and status values. Lookups distinguish usable capabilities from revoked, expired, or absent entries. A treaty store records proposed, active, suspended, revoked, expired, and consumed treaty state, and also retains pending cross-domain queries so that responses can be correlated with the originating treaty, peer serial, local serial, query identifier, and validity window.

These stores refine the abstract authorization predicate. Let $\text{CertStatus}(s, t)$, $\text{CapStatus}(d, t)$, and $\text{TreatyStatus}(id, t)$ denote the status of the certificate identified by serial s , the capability identified by digest d , and the treaty identified by id at time t . A request is admissible only if the certificate chain validates to the configured root, every status predicate on the path is active at t , the capability digest resolves to an active and unexpired capability, and any cross-domain request resolves to an active treaty that permits the requested predicate family and scope. This strengthens the security model because authorization cannot be satisfied by possession of a syntactically valid token alone.

12.2 Role-Gated Dispatch

The dispatch layer binds each message type to the roles permitted to originate or process that message. Certificate enrollment, certificate status changes, capability grants, object creation, object transfer, registry commits, anchor push and acknowledgement, treaty proposal, treaty co-signature, treaty revocation, treaty query forwarding, treaty query response, and error reporting are separate typed operations. The dispatcher rejects a message before handler execution when the local role, peer role, message type, tunnel state, or policy context is inconsistent. Handler-level checks then verify signatures, certificate status, capability status, treaty scope, query freshness, registry ownership, and anchor sequence.

Formally, let $\text{RoleOK}(r, m)$ be the implementation role matrix and let Handle_m be the message handler for message type m . For any message M of type m , UDIF requires

$$\text{Dispatch}(r, M, t) = \perp \quad \text{if} \quad \neg \text{RoleOK}(r, m).$$

If $\text{RoleOK}(r, m)$ holds, the corresponding handler is entered only under the authenticated tunnel context, and the handler remains responsible for message-specific semantic validation. This two-stage enforcement prevents unauthorized messages from consuming privileged handler paths and provides a clean reduction from authorization bypass to either dispatch-matrix failure or handler predicate failure.

12.3 Tunnel Record Validation

The implementation tunnel layer validates a UDIF record header before exposing the inner message. The canonical header is exactly 26 bytes and serializes a record-class flag, a 64-bit little-endian sequence number, a 64-bit little-endian timestamp, a 64-bit little-endian epoch counter, and an 8-bit suite identifier. Payload length is enforced by the enclosing QSTP packet length and by exact UDIF message decoding, not by a separate UDIF header field. The receiving side rejects replayed sequence numbers, reordered records, stale epochs after rekeying, suite mismatches, invalid class-to-message mappings, mutated inner headers, mutated inner messages, trailing bytes, malformed payloads, ciphertext mutation, authentication-tag mutation, and wrong-session ciphertexts. Rekey and close are explicit tunnel states rather than informal side effects.

This behavior is captured by the predicate $\text{RecordOK}(T, H, C, t)$ for tunnel state T , decoded header H , ciphertext C , and time t . The predicate is true only when the inner header is canonical, fresh, suite-compatible, flag-compatible with the decoded message type, epoch-compatible, exactly at the next receive sequence, completely decoded without trailing bytes, and authenticated by the underlying QSTP/RCS record protection. The receiver never advances the accepted sequence state for a rejected record. Thus a replay, reorder, malformed-record, or wrong-session attack cannot be amplified into state desynchronization without also defeating authenticated encryption or the header-validation predicate.

12.4 MCEL Ledgers and Checkpoints

The MCEL manager provides a commitment-based persistence layer over three ledgers: membership, registry, and transaction. Anchor evidence is not a fourth ledger but is derived by sealing and checkpointing these three. The manager supports distinct ledger selection, append-only record insertion, record reading, ledger-size queries, encrypted-record flags, manual block flushing, automatic block sealing, checkpoint creation, coordinated checkpoint groups across the three ledgers, and signed anchor creation. The storage backend rejects path traversal and maintains ledger isolation so that records written to one ledger are not addressable through another ledger type.

Let L_i be a ledger stream with append sequence i and let B_j be a sealed block containing a range of ledger records. A checkpoint commits to one or more sealed blocks, and an anchor commits to the checkpoint-derived roots plus counters. The formal anchor-chain proof therefore extends to MCEL persistence: an adversary who changes a prior record, moves a record across ledger namespaces, suppresses a record after block sealing, or fabricates a checkpoint must either find a hash collision, forge the checkpoint or anchor signature, or violate storage namespace separation.

12.5 Virtual-Network Conformance Model

UDIF defines a virtual-network conformance model in which a root, branch controllers, group controllers, and user agents are instantiated as independent logical roles, and the hierarchy is exercised through topology initialization, subordinate enrollment, certificate-status propagation, duplicate-serial rejection, and genesis-anchor creation. This model corresponds to the validation rules that the implementation handlers enforce. In particular, the hierarchy rejects invalid role transitions, unauthorized subordinate enrollment, duplicate serial numbers, inactive issuers, suspended or revoked parents, and anchors whose initial sequence or signature binding does not match the expected genesis state ($q = 0$).

The formal system model is therefore interpreted with the following executable constraint: every accepted state transition must be reproducible by the corresponding implementation handler under a valid entity context, active store status, authenticated tunnel context, canonical encoded message, and current time value. Conversely, any implementation rejection caused by malformed encoding, invalid status, unauthorized role, invalid treaty, failed signature, failed MAC, invalid Merkle proof, stale timestamp, or invalid sequence is a security-preserving rejection and not a protocol failure.

13 Limitations and Discussion

This section discusses the limitations of the UDIF security model, including modeling choices, trust assumptions on domains, and caveats for real world deployment. While the formal analysis covers the core cryptographic mechanisms, some classes of attacks are outside scope and must be addressed by implementation policy or environment constraints.

13.1 Modeling Limitations

The formal model assumes that all cryptographic primitives behave according to their idealized security assumptions. Several classes of attacks remain outside the present analysis.

Side channel leakage. The analysis assumes that all cryptographic implementations operate in constant time and do not leak key dependent information. Timing attacks, power analysis, and fault attacks must be mitigated at the implementation level.

Random number generation. The model assumes access to high quality randomness. Failures in random number generation can compromise key generation, signatures, and KEM security but are not modeled.

Domain trust assumptions. The analysis assumes correct Root behavior. A malicious or compromised Root can issue arbitrary certificates, override capabilities, and alter anchor histories. Root trust is a global assumption inherent to the UDIF hierarchy.

Physical access and endpoint compromise. UDIF does not protect against adversaries with physical access to a UA or GC endpoint. Compromise of endpoint state falls outside the protocol model and must be addressed by operational controls.

Out of band policy failures. Policies enforced by access masks and capabilities assume correct administrative behavior. Errors in policy configuration or human oversight are outside scope.

13.2 Deployment Considerations

The guarantees established in this paper are cryptographic; their realization in a fielded system depends on the following deployment boundaries, which should be treated as explicit assumptions during diligence and operations.

Transport boundary. UDIF’s confidentiality, integrity, handshake authentication, and ratchet rekeying are provided by the underlying QSTP channel. The transport guarantees of UDIF are therefore conditioned on the security of QSTP and on its correct configuration; UDIF contributes the record header, the sequence, epoch, and time-window discipline, and the trunk rekey schedule layered above it.

Ratchet scope. The trunk ratchet is symmetric and provides forward secrecy only. It does not provide post-compromise (future) security: an adversary that captures live session state can follow subsequent rekeys until a fresh authenticated session is established. Deployments requiring post-compromise recovery should schedule periodic full re-handshakes rather than relying on the symmetric ratchet.

Capability key custody. Capability tokens are authenticated with KMAC under the issuer key and are verifiable only by parties holding that key. The integrity of the capability system therefore depends on protecting issuer MAC keys; a compromised issuer key permits forgery of that issuer’s capabilities.

Storage integrity and durability. Tamper-evidence is cryptographic, but availability and rollback prevention depend on the durability and integrity of the local MCEL ledgers and key stores. Loss or truncation of a ledger before its state has been anchored upstream is detectable but not recoverable from cryptography alone, and must be addressed by storage-level durability and backup controls.

Operator and console controls. Server roles authenticate operators through a local console credential mechanism. This administrative layer is operational policy rather than part of the protocol’s cryptographic model, and must be hardened independently.

Time and randomness. Sequence, epoch, and window checks assume a roughly synchronized clock within the configured 60-second skew, and all key generation, signing, and encapsulation assume a high-quality entropy source. Failures in either are outside the protocol model.

14 Conclusion

This paper has presented a formal, game-based analysis of the Universal Digital Identity Framework. We modeled UDIF’s certificate hierarchy, capability delegation, canonical

encoding and domain-separated commitments, Merkle-committed registries, anchored audit chains, the policy-and-sequencing transport layer it operates over the QSTP channel, and the predicate-based minimal-disclosure query and treaty model. For each subsystem we gave precise security definitions and reductions: certificate and capability soundness to signature and MAC unforgeability; object, registry, and anchor integrity to collision resistance and signature unforgeability; transport confidentiality, integrity, and replay resistance to the security of the underlying authenticated channel together with UDIF's header and sequencing discipline; forward secrecy of trunks to the one-wayness of the symmetric ratchet derivation; and query minimal disclosure and cross-domain containment to commitment binding and capability unforgeability. A composition argument bounds the global adversarial advantage by the sum of the advantages against the underlying primitives. Scoped to the assumptions and deployment boundaries set out above, the analysis supports UDIF as a cryptographically rigorous framework for post-quantum identity, authorization, and cross-domain provenance, and provides a basis for subsequent independent cryptographic review.

References

- [1] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. U.S. Department of Commerce, 2015. <https://doi.org/10.6028/NIST.FIPS.202>.
- [2] National Institute of Standards and Technology. *SP 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*. U.S. Department of Commerce, 2016. <https://doi.org/10.6028/NIST.SP.800-185>.
- [3] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. U.S. Department of Commerce, 2024. <https://doi.org/10.6028/NIST.FIPS.203>.
- [4] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard*. U.S. Department of Commerce, 2024. <https://doi.org/10.6028/NIST.FIPS.204>.
- [5] National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard*. U.S. Department of Commerce, 2024. <https://doi.org/10.6028/NIST.FIPS.205>.
- [6] Bernstein, D. J., Chou, T., Lange, T., Niederhagen, R., Persichetti, E., Peters, C., Schwabe, P., Sendrier, N., Szefer, J., and Wang, W. *Classic McEliece: Conservative Code-Based Cryptography*. NIST Post-Quantum Cryptography Project submission. <https://classic.mceliece.org/>.
- [7] Bertoni, G., Daemen, J., Peeters, M., and Van Assche, G. *The Keccak Reference*. 2011. <https://keccak.team/keccak.html>.
- [8] Merkle, R. C. A digital signature based on a conventional encryption function. In *Advances in Cryptology - CRYPTO '87*, Lecture Notes in Computer Science 293, Springer, 1988.
- [9] Rogaway, P. Authenticated-encryption with associated-data. In *Proceedings of the 9th ACM Conference on Computer and Communications Security*, 2002.
- [10] Underhill, J. G. *Universal Digital Identity Framework (UDIF) Technical Specification*. Quantum Resistant Cryptographic Solutions Corporation, 2026. Revised specification accompanying this paper.

-
- [11] Underhill, J. G. *Quantum Secure Tunnelling Protocol (QSTP) Specification*. Quantum Resistant Cryptographic Solutions Corporation. https://qrcscorp.ca/documents/qstp_specification.pdf.
 - [12] Quantum Resistant Cryptographic Solutions Corporation. *QSTP Source Code Repository*. <https://github.com/QRCS-CORP/QSTP>.
 - [13] Underhill, J. G. *RCS Authenticated Stream Cipher and AEAD Construction*. Quantum Resistant Cryptographic Solutions Corporation. <https://www.qrcscorp.ca>.